

Replacement function

Document #: P2826R1
Date: 2023-11-05
Project: Programming Language C++
Audience: EWG
Reply-to: Gašper Ažman
<gasper.azman@gmail.com>

Contents

- [1 Introduction](#)
- [2 Status](#)
- [3 Proposal](#)
 - [3.1 Example 1: simple free function aliases](#)
 - [3.2 Example 2: simple member function aliases](#)
 - [3.3 Example 3: computed target](#)
 - [3.4 Example 4: deduce-to-baseclass](#)
 - [3.5 Discussion](#)
 - [3.6 What if the constant expression evaluates to null?](#)
- [4 Why this syntax](#)
- [5 Nice-to-have properties](#)
- [6 Related Work](#)
- [7 Use-cases](#)
 - [7.1 deduce-to-type](#)
 - [7.2 expression-equivalent for `std::strong\_order`](#)
 - [7.3 interactions with `\_\_builtin\_calltarget\(\)`](#)
 - [7.4 programmable UFCS](#)
 - [7.5 The “Rename Overload Set” / “Rename Function” refactoring becomes possible in large codebases](#)
 - [7.6 The “rename function” refactoring becomes ABI stable](#)
- [8 References](#)

§ 1 Introduction

This paper introduces a way to redirect a function call to a different function without a forwarding layer.

Example:

```
int g(int) { return 42; }
int f(long) { return 43; }
auto g(unsigned) = &f; // handle unsigned int with f(long)

int main() {
    g(1); // 42
    g(1u); // 43
}
```

One might think that declaring

```
int g(unsigned x) { return f(x); }
```

is equivalent; this is far from the case in general, which is exactly why we need this capability. See the motivation section for the full list of issues this capability solves.

First, however, this paper introduces what it *does*.

§ 2 Status

This paper is in early stages of exploration. The R1 will be presented to EWGI in Kona, 2023.

§ 3 Proposal

We propose a new kind of function definition of the form

function-body: ... = *constant-expression* ;

where the *constant-expression* evaluates to a pointer-to-function or pointer-to-member-function. We will call this function the **target**.

The associated function declaration has to be declared with a return type of `auto`, and cannot have been declared previously.

A call expression that resolves to a function thus declared instead resolves to the target. This may render the program ill-formed.

§ 3.1 Example 1: simple free function aliases

We could implement overload sets from the C library without indirection:

```
// <cmath>
auto exp(float) = &expf; // no duplication
double exp(double) { /* normal definition */ }
auto exp(long double) = &expl; // no duplication
```

This capability would make wrapping C APIs much easier, since we could just make overload sets out of individually-named functions.

§ 3.2 Example 2: simple member function aliases

```
template <typename T>
struct Container {
    auto cbegin() const -> const_iterator;
    auto begin() -> iterator;
    auto begin() const = &cbegin; // saves on template instantiations
};
```

§ 3.3 Example 3: computed target

`fmt::format` goes through great lengths to validate format string compatibility at compile time, but *really* does not want to generate code for every different kind of string.

This is extremely simple to do with this extension - just funnel all instantiations to a `string_view` overload.

```
template <typename FmtString, typename... Args>
    requires compatible<FmtString, Args...>
auto format(FmtString const&, Args const&... args)
    = static_cast<std::string*>(std::string_view, Args const&...)>
    (&format);
```

Contrast with the best we can realistically do presently:

```
template <typename FmtString, typename... Args>
    requires compatible<FmtString, Args...>
auto format(FmtString const& fmt, Args const&... args) -> std::string {
    return format(std::string_view(fmt), args...);
}
```

The second example results in a separate function for each format string (which is, say, one per log statement). The first provably never instantiates different function bodies for different format strings.

§ 3.4 Example 4: deduce-to-baseclass

Imagine we inherited from `std::optional`, and we wanted to forward operator*, but have it be treated the same as `value()`; note this is *far* easier to done with the help of `__builtin_calltarget` [P2825R0], so we'll use it here (it's just “give me the function pointer to the top-most call expression”):

```
template <typename T>
struct my_optional : std::optional<T> {
    template <typename Self>
        auto operator*(this Self&& self) =
            __builtin_calltarget(std::declval<Self>().value());
};
```

Without `__builtin_calltarget`, the computation becomes a *lot* uglier, because we need to conjure the function pointer type for each cv-ref qualification, so I'm not even going to include it here.

§ 3.5 Discussion

There are two parts such a function definition: the declarator, and the body.

The function signature as declared in the declarator participates in overload resolution. (the constant expression is not part of the immediate context).

If overload resolution picks this signature, then, *poof*, instead of substituting this signature into the call-expression, the replacement function is substituted. If this renders the program ill-formed, so be it.

§ 3.6 What if the constant expression evaluates to null?

That renders calling that overload ill-formed (and thus effectively means the same as a programmable = delete).

virtual, in which case you can now spell conditionally-implemented overrides. Yay. (Note: whether a class is abstract or not is decided when the vtable entry is tried to be synthesized and fails).

For virtual functions, though, the signature needs to match in the same way as signatures of overrides have to match the virtual interface declaration, otherwise we have a problem with ABI.

Example:

```
template <bool enable_implementations>
struct A {
    long g(int) { return 42; }
    long h(int) = enable_implementations ? &g : nullptr;
    virtual long f(int) = enable_implementations ? &g : nullptr;
};
struct Concrete : A<true> {};
struct Abstract : A<false> {};
struct Concrete2 : Abstract { long f(int) override { return 3; } };
void impl() {
    Concrete x; // ok
    x.h(2);     // ok, 42

    Concrete2 z; // ok
```

```
z.f(2);      // ok, 3
z.h(2);      // Error, h is explicitly deleted

Abstract y;  // Error, f(int) is abstract
};
```

§ 4 Why this syntax

- syntax noted actually already exist, just look for `=delete`
- it works for `0` as “not-provided” and `= delete` as “no body”, consistently

§ 5 Nice-to-have properties

- removal of library-defined dispatch from callstacks
- optimizers have to work way less hard
- major symbol reduction
- no issues with expressions - we don’t modify expressions (compare: parametric expressions)
- No extra work for overload resolution - this feature has no bearing on overload resolution, as the *constant-expression* is evaluated at declaration time.

§ 6 Related Work

Roughly related were Parametric Expressions [[P1221R1](#)], but they didn’t interact with overload sets very well.

This paper is strictly orthogonal, as it doesn’t give you a way to rewire the arguments, just substitute the function that’s actually invoked.

§ 7 Use-cases

There are many cases in c++ where we want to add a function to an overload set without wrapping it.

A myriad of use-cases are in the standard itself, just search for *expression_equivalent* and you will find many.

The main thing we want to do in all those cases is:

- step 1: detect a call expression pattern
- step 2: dispatch to the appropriate function for that case

The problem is that the detection of the pattern and the appropriate function to call often don’t fit into the standard overload resolution mechanism of C++, which necessitates a

wrapper, such as a customization point object, or some kind of dispatch function.

This dispatch function, however it's employed, has no way to distinguish prvalues from other rvalues and therefore cannot possibly emulate the copy-elision aspects of *expression-equivalent*. It is also a major source of template bloat. The proposed mechanism also solves a number of currently unsolved problems in the language (see the [use-cases section](#)).

§ 7.1 deduce-to-type

The problem of template-bloat when all we wanted to do was deduce the type qualifiers has plagued C++ since move-semantics were introduced, but gets much, worse with the introduction of (Deducing This) [[P0847R7](#)] into the language.

This topic is explored in Barry Revzin's [[P2481R1](#)].

This paper provides a way to spell this though a specific facility for this functionality might be desirable as a part of a different paper.

Observe:

```
struct A {};  
struct B : A {};  
  
template <typename T>  
auto _f_impl(T&& x) {  
    // only ever gets instantiated for A&, A&&, A const&, and A const&&  
};  
  
// TODO define cvref-derived-from  
template <typename D, typename B>  
concept derived_from_xcv = std::derived_from<std::remove_cvref_t<D>, B>;  
  
template <std::derived_from<A> T>  
auto f(T&&) = _f_impl<copy_cvref_t<T, A>>;  
  
void use_free() {  
    B b;  
    f(b); // OK, calls _f_impl(A&)  
    f(std::move(b)); // OK, calls _f_impl(A&&)  
    f(std::as_const(b)); // OK, calls _f_impl(A const&)  
}
```

Or, for an explicit-object member function:

```
struct C {  
    void f(this std::same_as<C> auto&& x) {} // implementation  
    template <typename T>  
    auto f(this T&& x) = static_cast<void (*)>(copy_cvref_t<T, C>&&)(f);  
    // or, with __builtin_calltarget (P2825) - helps if you just want to  
    // write the equivalent expression  
    template <typename T>
```

```

    auto f(this T&& x) = __builtin_calltarget(std::declval<copy_cvref_t<T,
        C>&&>().f());
};
struct D : C {};

void use_member() {
    D d;
    d.f();           // OK, calls C::f(C&)
    std::move(d).f(); // OK, calls C::f(C&&)
    std::as_const(d).f(); // OK, calls C::f(C const&)
}

```

§ 7.1.1 Don't lambdas make this shorter?

Observe that we could also have just defined the inline lambda as the *constant-expression*:

```

struct A {};
struct B : A {};

// saves on typing?
template <std::derived_from<A> T>
auto f(T&&) = +[](copy_cvref_t<T, A>&&) {};

void use_free() {
    B b;
    f(b);           // OK, calls f-lambda(A&)
    f(std::move(b)); // OK, calls f-lambda(A&&)
    f(std::as_const(b)); // OK, calls f-lambda(A const&)
}

```

While at first glance this saves on template instantiations, it's not really true (but it might save on executable size if the compiler employs COMDAT folding).

The reason is that the lambdas still depend on τ , since one can use it in the lambda body.

This is very useful if we need to manufacture overload sets of *functions with identical signatures*, such as needed for type-erasure.

Continued example:

```

template <typename T>
int qsort_compare(T const*, T const*) = +[](void const* x, void const* y)
{
    return static_cast<int>(static_cast<T const*>(x) <=> static_cast<T
        const*>(y));
    //                                     ^^
};

// special-case c-strings
int qsort_compare(char const* const*, char const* const*) = +[](void
    const* x, void const* y) {
    auto const x_unerased = static_cast<char const* const*>(x);
    auto const y_unerased = static_cast<char const* const*>(y);
    return strcmp(*x_unerased, *y_unerased);
}

```

```
};

void use_qsort_compare() {
    std::vector<int> xs{1, 4, 3, 5, 2};
    ::qsort(xs.data(), xs.size(), sizeof(int),
            __builtin_calltarget(qsort_compare(std::declval<int*>(),
            std::declval<int*>())));
    std::vector<char const*> ys{"abcd", "abdc",
    "supercalifragilisticexpialidocious"};
    ::qsort(ys.data(), ys.size(), sizeof(char const*),
            __builtin_calltarget(qsort_compare("", "")));
}
```

So, while lambdas don't turn out to be a useful "template-bloat-reduction" strategy, they *do* turn out to be a very useful strategy for type-erasure.

§ 7.2 expression-equivalent for `std::strong_order`

`std::strong_order` needs to effectively be implemented with an `if constexpr` cascade (link to `libc++`'s implementation).

This function template would probably be much easier to optimize for a compiler if it were factored as follows:

```
inline constexpr auto struct {
    template <typename T, typename U>
    static constexpr auto __select_implementation<T, U>() { /* figure out
    the correct function to call and return the pointer to it */ }
    template <typename T, typename U>
    static auto operator(T&&, U&&) = __select_implementation<T, U>();
} strong_order;
```

Implementing it like that allows one to dispatch to functions that don't even take references for built-in types, eliding not just a debug/inlining scope, but also the cleanup of references.

It allows one to dispatch to ABI-optimal versions of the parameter-passing strategy. It lets us make function objects truly invisible to debuggers.

§ 7.3 interactions with `__builtin_calltarget()`

While [P2826R0] makes this paper much easier to utilize, in the presence of this paper, it becomes truly invaluable. Computing the calltarget while having to know the exact function pointer type of the called function is very difficult, and needless, since the compiler is very good at resolving function calls.

§ 7.4 programmable UFCS

If we get a new revision of [P1214R0], (`std::invoke-in-the-language`), one could substitute a free function call with a pointer-to-member-function, or vice versa, thus

enabling both call syntaxes as desired.

§ 7.5 The “Rename Overload Set” / “Rename Function” refactoring becomes possible in large codebases

In C++, “rename function” or “rename overload set” are not refactorings that are physically possible for large codebases without at least temporarily risking overload resolution breakage.

However, with this paper, one can disentangle overload sets by leaving removed overloads where they are, and redirecting them to the new implementations, until they are ready to be removed.

(*Reminder:* conversions to reference can lose fidelity, so trampolines in general do not work).

One can also just define

```
auto new_name(auto&&...args)
    requires { __builtin_calltarget(old_name(FWD(args)...))(FWD(args)...)}
    = __builtin_calltarget(old_name(FWD(args)...));
```

and have a new name for an old overload set (as resolved at the declaration of `new_name`).

§ 7.6 The “rename function” refactoring becomes ABI stable

- we can finally move overload sets around and not break ABI in some cases since we basically gain function aliases.

§ 8 References

[P0847R7] Barry Revzin, Gašper Ažman, Sy Brand, Ben Deane. 2021-07-14. Deducing this.

<https://wg21.link/p0847r7>

[P1214R0] JeanHeyd Meneide. 2018-10-06. Pointer to Member Functions and Member Objects are just Callables!

<https://wg21.link/p1214r0>

[P1221R1] Jason Rice. 2018-10-03. Parametric Expressions.

<https://wg21.link/p1221r1>

[P2481R1] Barry Revzin. 2022-07-15. Forwarding reference to specific type/template.

<https://wg21.link/p2481r1>

[P2825R0] Gašper Ažman. 2023-03-15. `calltarget(unevaluated-call-expression)`.

<https://wg21.link/p2825r0>

[P2826R0] Gašper Ažman. 2023-03-15. Replacement functions.

<https://wg21.link/p2826r0>